Permutation

The Pharaohs use the relative movement and gravity of planets to accelerate their spaceships. Suppose a spaceship will pass by n planets with orbital speeds $p[0], p[1], \ldots, p[n-1]$ in order. $(p[0], p[1], \ldots, p[n-1])$ is a permutation of $0, 1, \ldots, n-1$. For each planet, the Pharaohs scientists can choose whether to accelerate the spaceship using this planet or not. To save energy, after accelerating by a planet with orbital speed $p[i]$, the spaceship cannot be accelerated using any planet with orbital speed $p[j] < p[i]$. In other words, the chosen planets form an **increasing subsequence** of $p[0], p[1], \ldots, p[n-1]$.

The scientists have identified that there are a total of k different ways a set of planets can be chosen to accelerate the spaceship but they have lost their record of all the orbital speeds. Your task is to recover the sequence $p[0], p[1], \ldots, p[n-1]$.

You will need to solve the problem for q different spaceships. For each spaceship i, you will get an integer k_i, representing the number of different ways a set of planets can be chosen to accelerate the spaceship. Your task is to find a sequence of orbital speeds with the smallest possible length n_i such that there are exactly k_i ways a subsequence of planets with increasing orbital speeds can be chosen.

Implementation details

You should implement the following procedure:

```
int[] construct_permutation(int64 k)
```

- k: is the desired number of increasing subsequences.
- This procedure should return an array of n elements where each element is between 0 and $n-1$ inclusive.
- The returned array must be a valid permutation having exactly k increasing subsequences.
- This procedure is called a total q of times. Each of these calls should be treated as a separate scenario.

Constraints

- $1 \leq q \leq 100$
- $2 \leq k_i \leq 10^{18}$ (for all $0 \leq i \leq q-1$)

Subtasks

1. (10 points) $2 \leq k_i \leq 90$ (for all $0 \leq i \leq q-1$)
2. (90 points) No additional constraints. For this subtask, let m be the maximum permutation length you used in any scenario. Then, your score is calculated according to the following table:

Condition	Score
$m \leq 90$	90
$90 < m \leq 120$	$90 - (m - 90)$
$120 < m \leq 5000$	$60 - \frac{(m-120)}{90}$
$m > 5000$	0

Example

Example 1

Consider the following call:

```
construct_permutation(3)
```

This procedure should return a permutation with exactly 3 increasing subsequences. A possible answer is $[1, 0]$, which has $[]$ (empty subsequence), $[0]$ and $[1]$ as increasing subsequences.

Example 2

Consider the following call:

```
construct_permutation(8)
```

This procedure should return a permutation with exactly 8 increasing subsequences. A possible answer is $[0, 1, 2]$.

Sample grader

The sample grader reads the input in the following format:

- line 1: q
- line $2 + i$ ($0 \leq i \leq q - 1$): k_i

The sample grader prints a single line containing the return value of `construct_permutation`, or an error message if one has occurred.